Architectures parallèles

Chapitre 3

Multithreading

Multithreading

RAPPEL C/C++

La notion d’adresse

- Tout élément (variable, constante, instructions,…) manipulé par l'ordinateur est stocké dans sa mémoire.

- Cette mémoire est constituée d'une série de « cases »

- Pour avoir accès à un élément, il faut connaître le numéro de cette case : -> **l’adresse**

- Toute case mémoire a une adresse unique (sur 32 ou 64 bits).

L'adresse d'une variable

```
#include <iostream.h>
#include <stdio.h>

int main() {

    int toto=45;

    cout<<toto<<endl;        //affiche la valeur 45
    printf("%d \n",toto);    //affiche la valeur 45

    cout<<&toto<<endl;        //affiche l'adresse de toto
    printf("%p \n",&toto);    //affiche l'adresse de toto

    return 0;
}
```

Le pointeur

Une adresse est une valeur.

On peut stocker cette valeur dans une variable : les **pointeurs** sont des variables qui contiennent des adresses

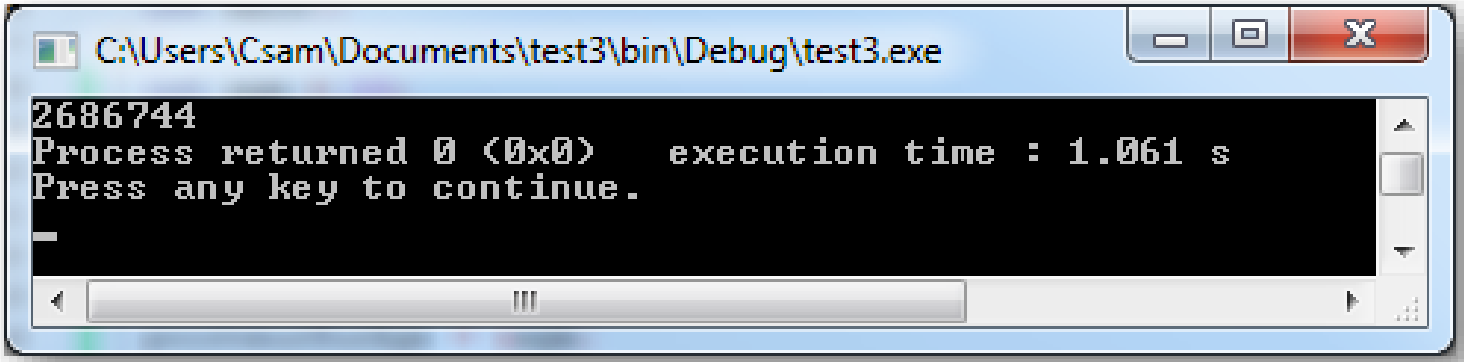
On dit que **le pointeur pointe sur la variable pointée**

La valeur d'un pointeur peut changer. *Cela ne signifie pas que la variable pointée est déplacée en mémoire, mais plutôt que le pointeur pointe sur autre chose.*

Les pointeurs disposent d'un type (qui correspond généralement au type de la variable pointée)

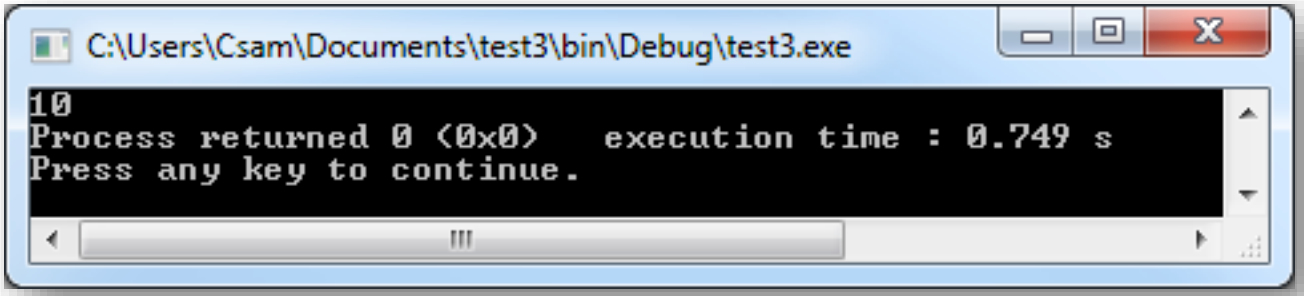
Le pointeur

```
int age = 10;
//  "Je crée un pointeur"
int *pointeurSurAge;
// pointeurSurAge contient l'adresse de la variable age
pointeurSurAge = &age;

printf("%d", pointeurSurAge);
```



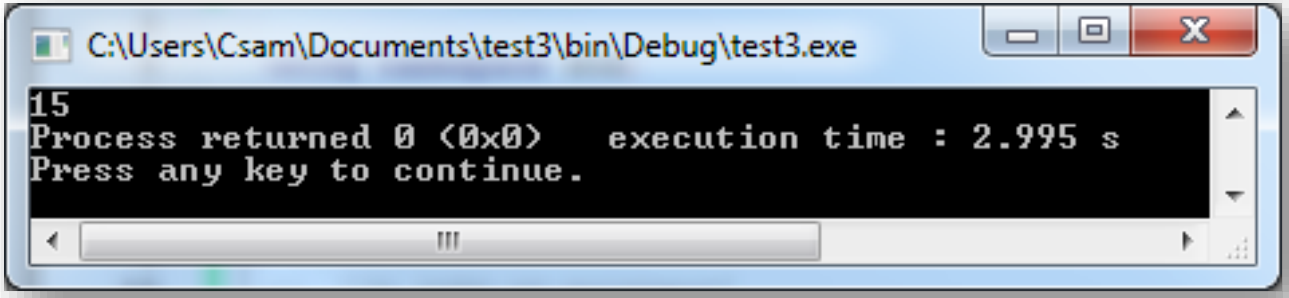
Le pointeur

```
int age = 10;
//  "Je crée un pointeur"
int *pointeurSurAge;
// pointeurSurAge contient l'adresse de la variable age
pointeurSurAge = &age;


printf("%d", *pointeurSurAge);
```



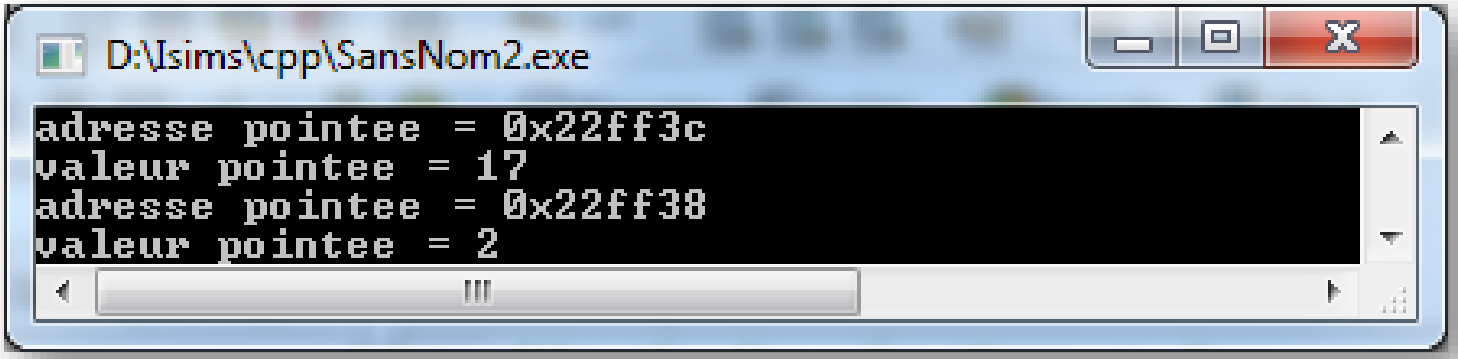
Le pointeur

```
int age = 10;
//  "Je crée un pointeur"
int *pointeurSurAge;
// pointeurSurAge contient l'adresse de la variable age
pointeurSurAge = &age;

*pointeurSurAge+=5;

printf("%d", age);
```



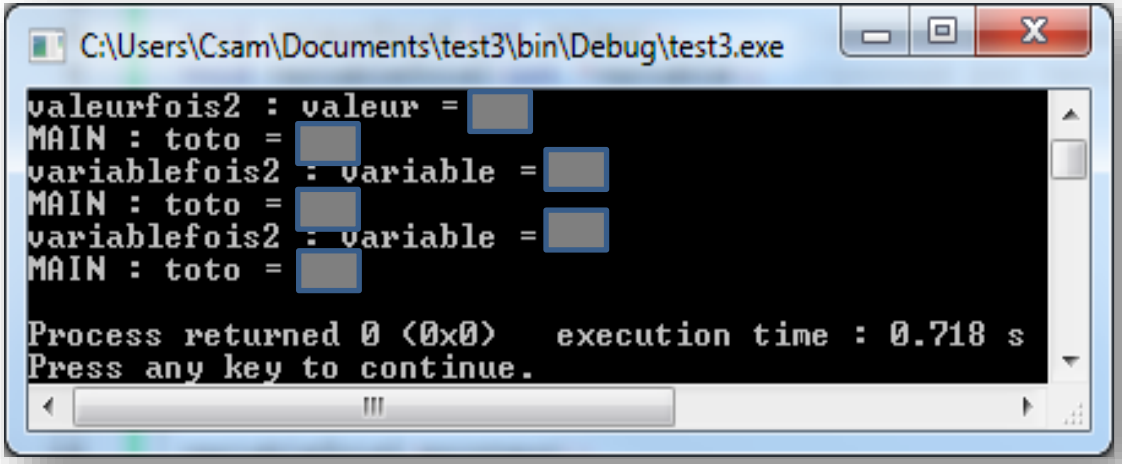
Le pointeur

```
int a=17,b=2;
int *pointeur=&a;

cout<<"adresse pointee = "<<pointeur<<endl;
cout<<"valeur pointee = "<<*pointeur<<endl;

/*on peut décider de changer l'adresser stockée par le pointeur pour qu'il
pointe ailleurs (pas possible avec une référence) */

pointeur=&b;

cout<<"adresse pointee = "<<pointeur<<endl;
cout<<"valeur pointee = "<<*pointeur<<endl;
```



Passage par valeur vs variable

```
void valeurfois2(int valeur)        //passage par valeur
{
    valeur*=2;
    cout<<"valeurfois2 : valeur = "<<valeur<<endl;
}

void variablefois2(int *variable)  //passage par variable
{
    *variable*=2;
    cout<<"variablefois2 : variable = "<<*variable<<endl;
}

int main()
{
int toto = 10;
int *pointeur=&toto;

valeurfois2(toto);
cout<<"MAIN : toto = "<<toto<<endl;

variablefois2(pointeur);
cout<<"MAIN : toto = "<<toto<<endl;

variablefois2(&toto);
cout<<"MAIN : toto = "<<toto<<endl;

    return 0;
}
```



Le pointeur : NE JAMAIS OUBLIER 1

Sur une variable :

age : valeur de la variable age

&age : l'adresse où se trouve la variable age

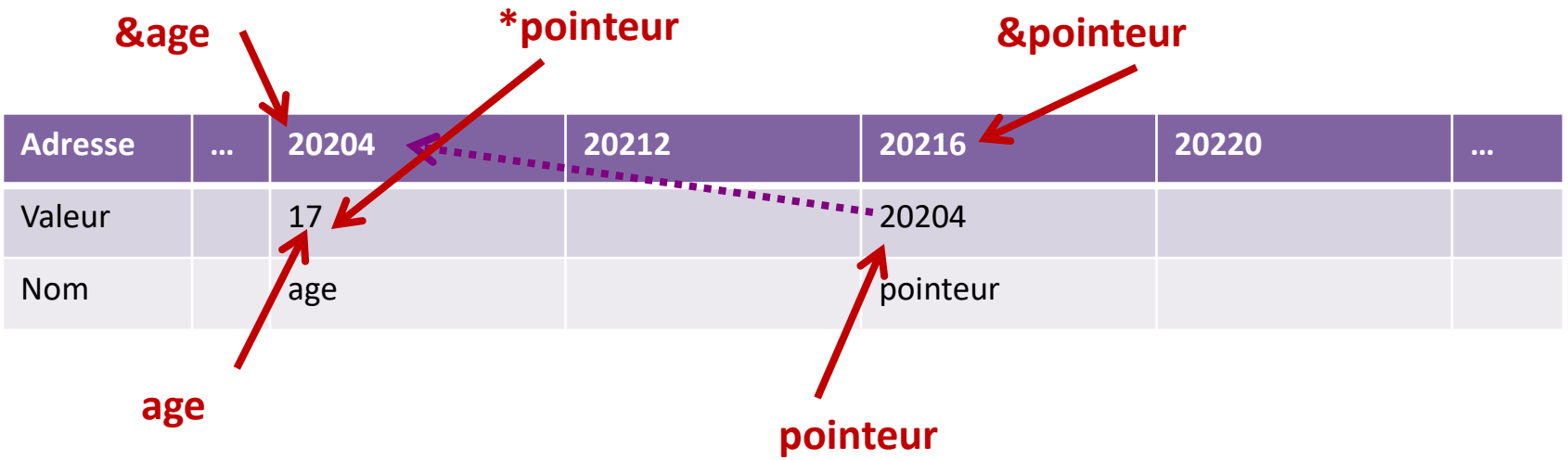
Sur un pointeur :

pointeur : la valeur de pointeurSurAge
(cette valeur étant une adresse).

***pointeur** : la valeur de la variable qui se trouve à l'adresse contenue dans pointeurSurAge

Le pointeur : NE JAMAIS OUBLIER 2

```
int age=17;
int *pointeur=&age;
```



Allocation dynamique

Il y a 2 façons de créer une variable (allouer de la mémoire) :

Automatiquement (= statiquement) :

```
int toto;
```

C'est la méthode que vous connaissez et que vous utilisez.

Manuellement (= dynamiquement) :

La mémoire est allouée pendant l'exécution du programme

Les variables sont créées à la volée

Allocation dynamique

Il faut inclure la bibliothèque <stdlib.h>

- **malloc** : demande au système d'exploitation la permission d'utiliser de la mémoire.

- **free** : la place en mémoire est libérée,

Les 3 étapes de l'allocation dynamique :

- **malloc** pour demander de la mémoire

- **Vérifier la valeur retournée par malloc** pour savoir si l'OS a bien réussi

- Une fois qu'on a fini d'utiliser la mémoire, on doit la libérer avec **free**.

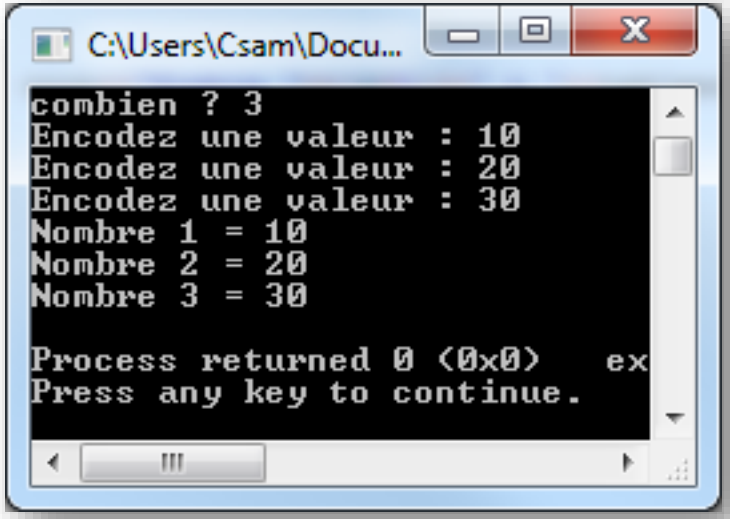
Allocation dynamique

Nombre d'OCTET à allouer

```
int main()
{
    int* memoireAllouee = NULL;

    memoireAllouee = (int*) malloc( sizeof(int) );

    if (memoireAllouee == NULL) // On vérifie si la mémoire a été allouée
        {
            cout<<"ERREUR : malloc"<<endl;
            exit(0); // Erreur : on arrête tout !
        }
    // On peut utiliser ici la mémoire
    cout<<"Encodez un nombre :"<<endl;
    cin>>*memoireAllouee;
    cout<<"Le nombre : "<<*memoireAllouee<<endl;

    free(memoireAllouee);
    // On n'a plus besoin de la mémoire, on la libère

    return 0;
}
```

Allocation dynamique d'un tableau

```
int nombre=0;

cout<<"combien ? ";

cin>>nombre;
int *tableauDYN=NULL;

tableauDYN=(int*)malloc(nombre*sizeof(int));


for(int i=0;i<nombre;i++)
{
    cout<<"Encodez une valeur : ";
    cin>>tableauDYN[i];
}

for(int i=0;i<nombre;i++)
        cout<<"Nombre "<<(i+1)<<" = "<<tableauDYN[i]<<endl;

free(tableauDYN);
```



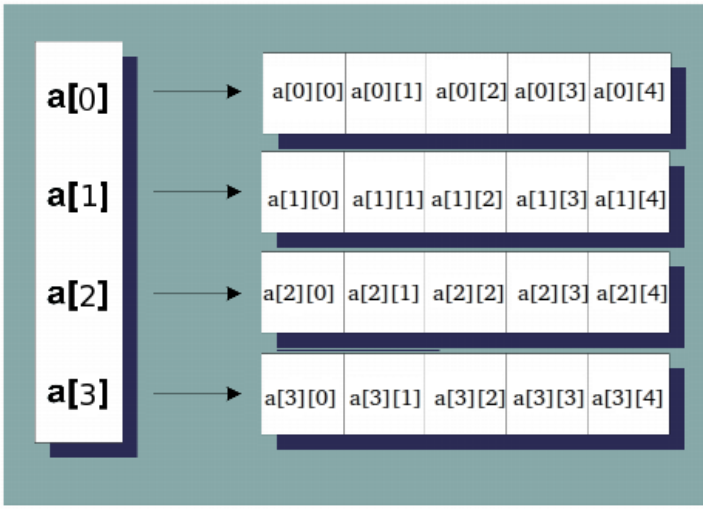
Allocation dynamique en C++

```
int *pointeur=NULL;

pointeur = new int;          // Allocation (le malloc du C)

delete pointeur;             //On libère la case mémoire (le free du C)

//Tableau Dynamique :

int taille=99;
int *TabDyn=NULL;

TabDyn = new int[taille];

delete[] TabDyn;
```

Nombre d'ELEMENT à allouer

pointeurs multiples et allocation dynamique

```
// crée dynamiquement un tableau à 2D en C++

int **ary;      //double pointeur

ary = new int*[sizeY]; //allocation d'un vecteur de pointeurs


//allocation des cases pour le stockage des données
for(int i = 0; i < sizeY; ++i) {
    ary[i] = new int[sizeX];
}

…
//désallocation
for(int i = 0; i < sizeY; ++i) {
    delete [] ary[i];
}
delete [] ary;
```



Allocation dynamique

Ne pas mélanger **malloc** (C) et **delete** (C++) avec un même pointeur

Ne pas mélanger **new**(C++) et **free** (C) avec un même pointeur

Toujours libérer la mémoire

sinon **perte de mémoire**

Bien initialiser les pointeurs à NULL

sinon **« pointeurs fous »**

Erreur de segmentation

Segmentation Fault

est un plantage d'une application qui a tenté d'accéder à un emplacement mémoire qui ne lui était pas alloué

Avant d’appeler à l’aide,
tu vérifies tes pointeurs !

TP Matrix 1 : multiplication matricielle

Veuillez réaliser un programme qui :

1. demande à l'utilisateur une valeur entière N
2. Alloue dynamiquement 3 matrices (A, B et C) d'entiers et de taille N x N
3. Remplit les matrices A et B de valeurs (aléatoires ou non)
4. Effectue la multiplication matricielle C=A*B
5. Affiche la durée qui a été nécessaire pour le calcul (et exclusivement pour le calcul)
6. N'oubliez pas de désallouer

Conservez bien votre programme, car vous allez en avoir besoin plus tard (dossier à réaliser)

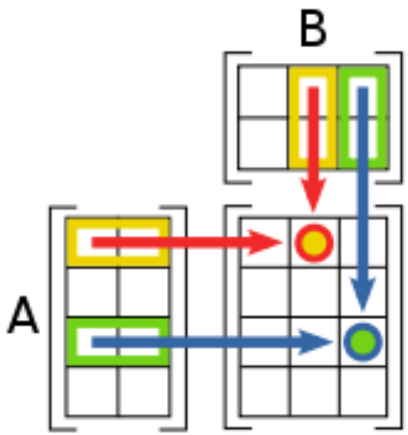
Si ce programme n'est pas fonctionnel

et/ou

vous n'arrivez pas à réaliser ce programme par vous-même

La réalisation des manips suivantes vous sera impossible

Exercice 1 : multiplication matricielle



$$c_{12} = \sum_{r=1}^{2} a_{1r} b_{r2} = a_{11} b_{12} + a_{12} b_{22}$$

$$c_{33} = \sum_{r=1}^{2} a_{3r} b_{r3} = a_{31} b_{13} + a_{32} b_{23}$$

$$\begin{pmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \end{pmatrix} \times \begin{pmatrix} 7 & 8 \\ 9 & -1 \\ -2 & -3 \end{pmatrix} = \begin{pmatrix} 1\times 7 + 2\times 9 - 3\times 2 & 1\times 8 - 2\times 1 - 3\times 3 \\ 4\times 7 + 5\times 9 - 2\times 6 & 4\times 8 - 5\times 1 - 6\times 3 \end{pmatrix}$$

$$= \begin{pmatrix} 19 & -3 \\ 61 & 9 \end{pmatrix}$$

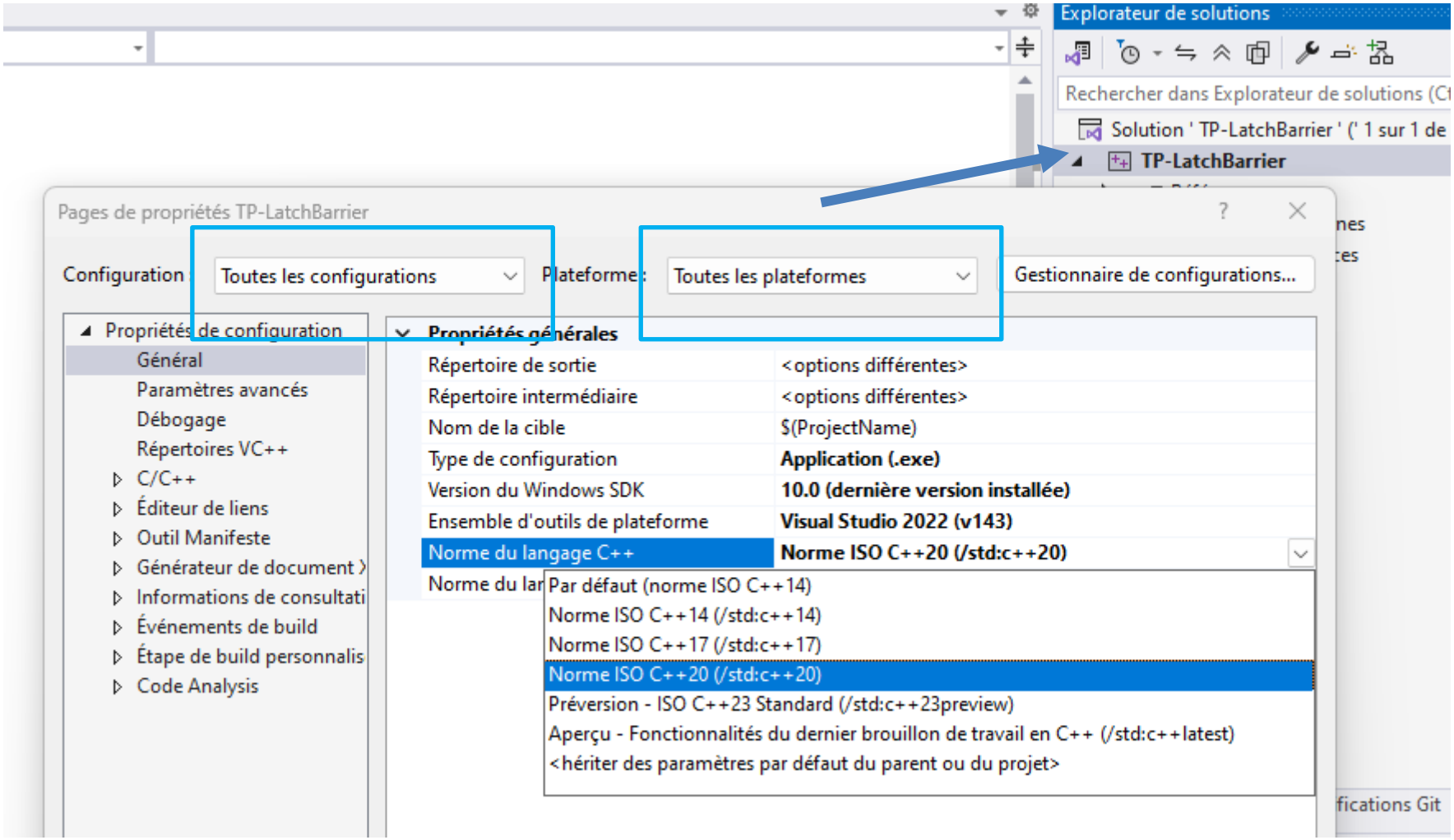
Mesurer le temps

```
#include <chrono>

std::chrono::steady_clock::time_point start, end;

start = std::chrono:: steady_clock::now();

    //job job job

end = std::chrono:: steady_clock::now();

std::chrono::steady_clock::duration duration = stop - start;

std::cout << std::chrono::duration_cast<chrono::microseconds>(duration).count();
```

Multithreading

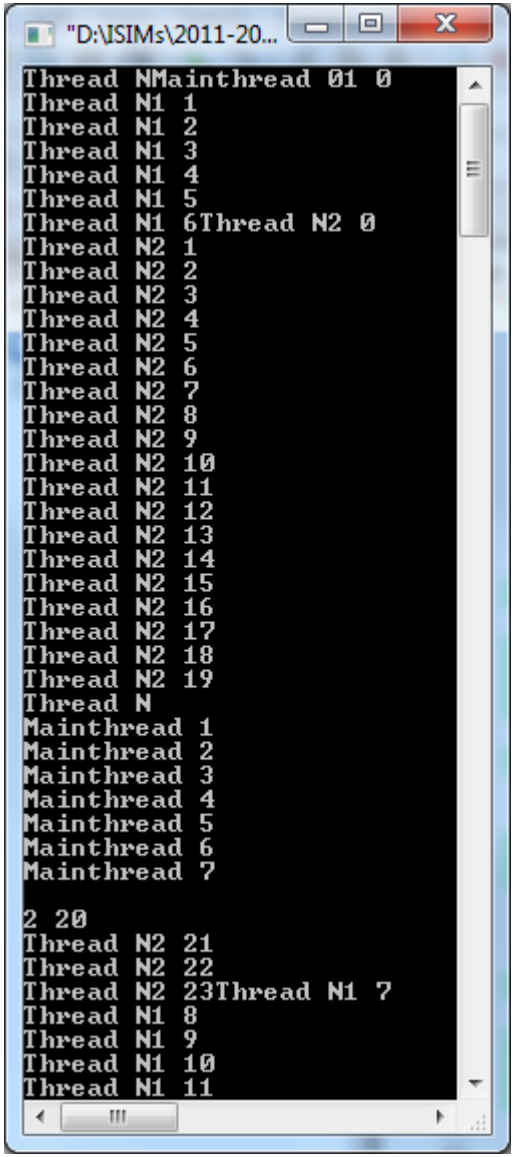
MULTITHREADING EN C++ 20

Choisir la bonne version de C++



Premier programme multithread

```
1   #include <iostream>
2   #include <thread>
3
4   using namespace std;
5
6   void fonction(int id)
7   {
8       for (int i = 0; i < 50; i++)
9       {
10          cout << "Thread N" << id << " " << i << endl;
11      }
12
13  }
14
15  int main()
16  {
17      thread t1(fonction, 1);
18      thread t2(fonction, 2);
19
20      for (int i = 0; i < 50; i++)
21      {
22          cout << "Mainthread " << i << endl;
23      }
24
25
26
27
28      return 0;
29  }
```



Jthread (1)

```
1  #include <iostream>
2  #include <thread>
3
4  void fct()
5  {
6      int x=0;
7      while (1)
8      {
9          x++;
10         std::cout << x << std::endl;
11     }
12 }
13
14
15 int main()
16 {
17     std::thread t(fct);
18
19     std::cout << "attente du thread" << std::endl;
20     t.join();
21     std::cout << "fin du programme" << std::endl;
22     return 0;
23 }
```

Ne termine jamais

Jthread (2)

```cpp
#include <iostream>
#include <thread>

void fct()
{
    int x=0;
    while (1)
    {
        x++;
        std::cout << x << std::endl;
    }
}


int main()
{
    std::jthread t(fct); //<---- ici

    std::cout << "attente du thread" << std::endl;
    t.request_join();   //<---- et là
    t.join();
    std::cout << "fin du programme" << std::endl;
    return 0;
}
```

Ne termine toujours pas

Jthread (3)

```cpp
#include <iostream>
#include <thread>

void fct(std::stop_token st)
{
    int x=0;
    while(!st.stop_requested())
    {
        x++;
        std::cout << x << std::endl;
    }
}

int main()
{
    std::jthread t(fct);

    std::cout << "attente du thread" << std::endl;
    t.requested_join();
    t.join();
    std::cout << "fin du programme" << std::endl;
    return 0;
}
```

Ok le thread se coupe
et libère automatiquement
ses ressources

Les ressources statiques !

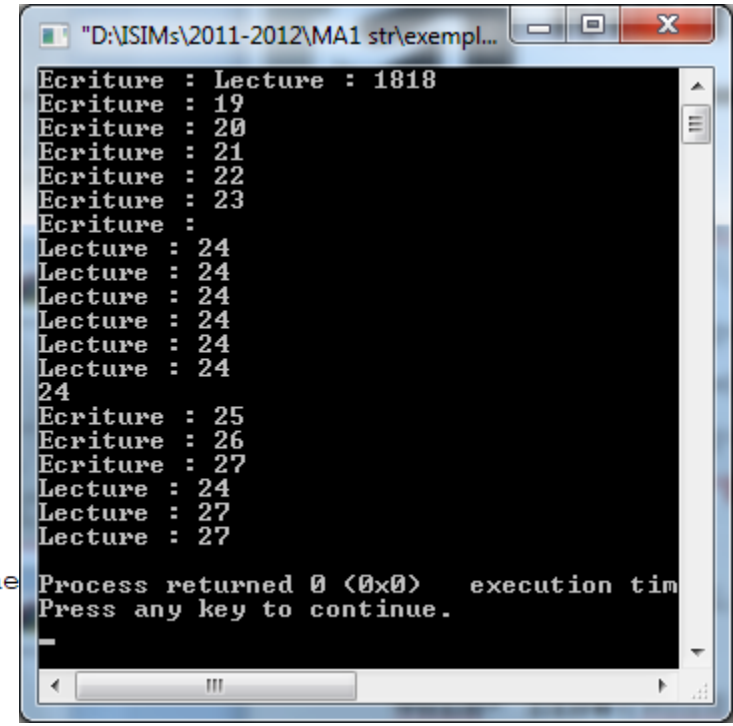
Attention aux ressources
dynamiquement créées
avec des pointeurs explicites

Multithreading avec C++

DATA RACE

Partage de ressources

```
1   #include <iostream>
2   #include <thread>
3
4   using namespace std;
5
6   void ecrire(int *donnee)
7   {
8       for (int i = 0; i<10; i++)
9       {
10          *donnee += 1;
11          cout << "Ecriture : " << *donnee << endl;
12      }
13  }
14
15  void lire(int *donnee)
16  {
17      for (int i = 0; i<10; i++) cout << "Lecture : " << *donne
18  }
19
20
21  int main()
22  {
23      int valeur = 17;
24      thread t_ecriture(ecrire, &valeur);
25      thread t_lecture(lire, &valeur);
26
27      t_ecriture.join();
28      t_lecture.join();
29
30      return 0;
31  }
```



Partage des ressources et exclusion mutuelle

Dans l'illustration suivante, le résultat n'est pas prévisible :

Thread A	Thread B
1A: Lire variable V	1B: Lire variable V
2A: Add 1 à la variable V	2B: Add 1 à la variable V
3A: Écrire la variable V	3B: Écrire la variable V

Lire, Lire, Add, Add, Écrire, Écrire -> V+1

Lire, Add, Écrire, Lire, Add, Écrire -> V+2

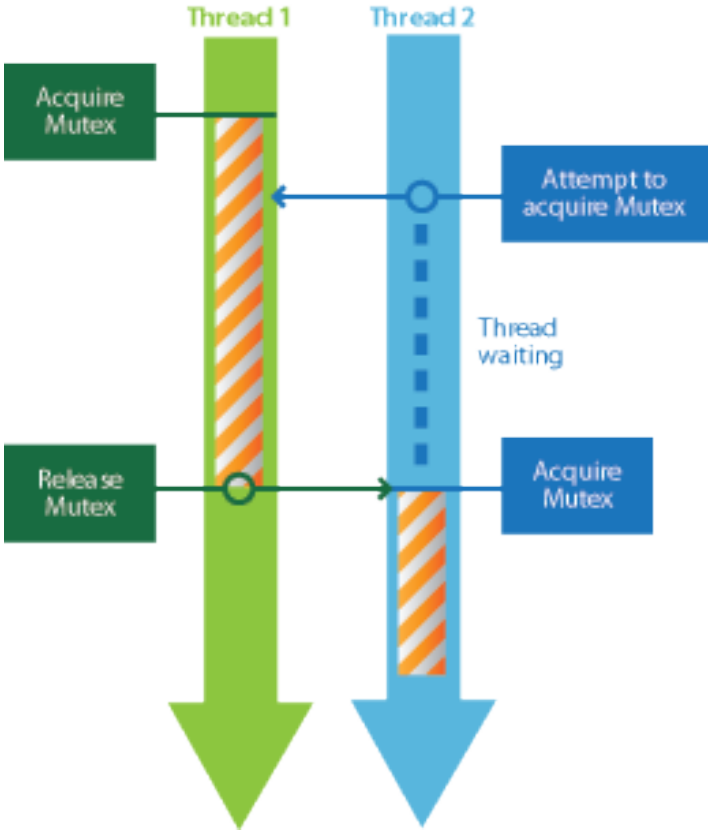
Ce phénomène est connu sous le nom de **data race** **(situation de compétition)**

Solution :

Thread A	Thread B
1A: Verrouiller la variable V	1B: Verrouiller la variable V
2A: Lire la variable V	2B: Lire la variable V
3A: Add 1 à la variable V	3B: Add 1 à la variable V
4A: Écrire la variable V	4B: Écrire la variable V
5A: déverrouiller la variable V	5B: déverrouiller la variable V

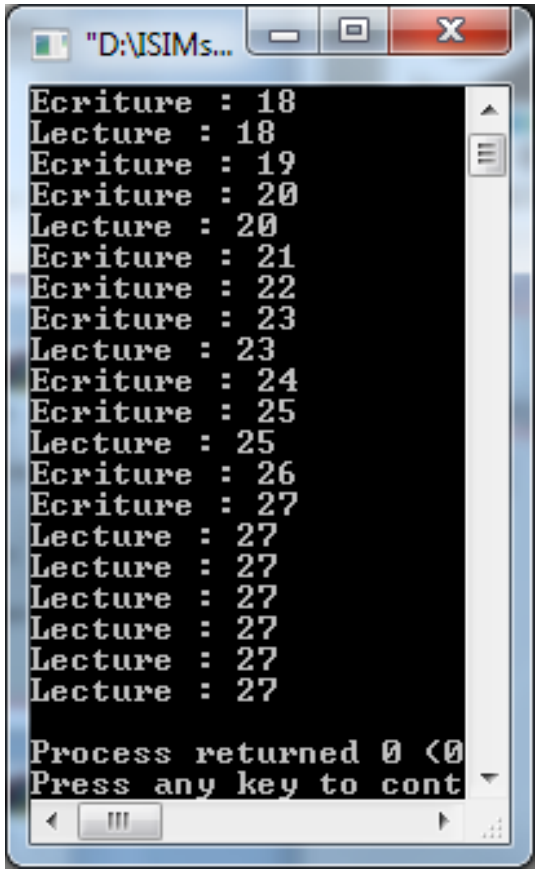
Le Mutex (1)

Pour se prémunir contre ce problème, on doit délimiter **une section critique**



Le Mutex (2)

```cpp
#include <iostream>
#include <thread>
#include <mutex>

std::mutex mtx;

void ecrire(int *donnee)
{
    for (int i = 0; i<10; i++)
    {
        mtx.lock();
        *donnee += 1;
        std::cout << "Ecriture : " << *donnee << std::endl;
        mtx.unlock();
    }
}

void lire(int *donnee)
{
    for (int i = 0; i < 10; i++)
    {
        mtx.lock();
        std::cout << "Lecture : " << *donnee << std::endl;
        mtx.unlock();
    }
}

int main()
{
    int valeur = 17;
    std::thread t_ecriture(ecrire, &valeur);
    std::thread t_lecture(lire, &valeur);

    t_ecriture.join();
    t_lecture.join();

    return 0;
}
```



Interblocage

Une situation de blocage (2 tâches s'attendent mutuellement) est appelé **étreinte fatale** ou **interblocage (deadlock).**

TâcheA :	TâcheB :
Obtenir M1	Obtenir M2
Obtenir M2	Obtenir M1
Action nécessitant les deux verrous	Action nécessitant les deux verrous
Rendre M2	Rendre M1
Rendre M1	Rendre M2

Solution préventive :

acquérir les mutex dans le même ordre.

Départ volontaire

```
5   std::mutex mtx;
6
7   void ecrire(int *donnee)
8   {
9       for (int i = 0; i<10; i++)
10      {
11          mtx.lock();
12          *donnee += 1;
13          std::cout << "Ecriture : " << *donnee << std::endl;
14          return;
15          mtx.unlock();
16      }
17  }
18
19  void lire(int *donnee)
20  {
21      for (int i = 0; i < 10; i++)
22      {
23          mtx.lock();
24          std::cout << "Lecture : " << *donnee << std::endl;
25          mtx.unlock();
26      }
27  }
28
29  int main()
30  {
31      int valeur = 17;
32      std::thread t_ecriture(ecrire, &valeur);
33      std::thread t_lecture(lire, &valeur);
34
35      t_ecriture.join();
36      t_lecture.join();
37
38      return 0;
39  }
```

Situation de blocage !!!

Mutex et section critique

```
7   void ecrire(int *donnee)
8   {
9       for (int i = 0; i<10; i++)
10      {
11          mtx.lock();
12          *donnee += 1;
13          std::cout << "Ecriture : " << *donnee << std::endl;
14          mtx.unlock();
15      }
16  }
```

Section critique

Il ne faut pas confondre l'utilisation d'un mutex avec une section critique

Un mutex est un mécanisme logiciel qui permet d'implémenter une section critique mais il existe d'autres façons de faire

Attention à ne pas abuser des sections critiques car cela risque de "séquentialiser" l'exécution

Mutex : syntaxe alternative

```
7   void ecrire(int *donnee)
8   {
9       for (int i = 0; i<10; i++)
10      {
11
12          std::lock_guard<std::mutex> lock(mtx);
13          *donnee += 1;
14          std::cout << "Ecriture : " << *donnee << std::endl;
15
16      }
17  }
```

Section critique

Libère automatiquement le mutex une fois arrivé ici

Performances des mutex

Un mutex est une primitive logicielle

Son exécution peut varier d'une cible à l'autre
(matériel / OS / compilateur)

Dans le meilleur des cas, le thread en attente d'un mutex reste actif sur le CPU jusqu'à ce qu'il puisse poursuivre (rare)

Dans le pire des cas (le plus courant), le thread passe en veille et est déchargé du CPU. Il devra donc être rechargé complètement quand le mutex sera libéré.

De plus, la libération d'un mutex n'est pas forcément synonyme de réveil des threads en attente du mutex.

Ce point peut être important en fonction du contexte et **du grain de parallélisme**

Acquisition non bloquante du mutex

```
bool ok=false;

while(!ok)
{
    if(le_mutex.try_lock())
    {
        //ok j'ai le mutex
        ok=true;
        le_mutex.unclock();
    }
    else
    {
        //je m'occupe
    }
}
```

Variante avec un "timed_mutex"

```
std::timed_mutex mutex;
```

```
bool ok=false;

while(!ok)
{
    if(mutex.try_lock_for(chrono::milliseconds(100)))
    {
        //ok j'ai le mutex
        ok=true;
        mutex.unclock();
    }
    else
    {
        //je m'occupe
    }
}
```

Il existe aussi *try_lock_until(l'heure)*

…

Le mutex partagé

std::shared_mutex **pastouche**;

Peut être verrouillé comme un mutex classique

ou

verrouillé en mode partagé.

Le mode partagé n'est pas un verrou exclusif
Les accès (partagés) peuvent être concurrents

Pendant qu'un mutex partagé est verrouillé de façon exclusive
il ne pourra pas être verrouillé de façon partagée

et vice versa

Existe en version "timed"

```
//Écrivain

pastouche.lock();
            Écriture
pastouche.unlock();
```

```
//Lecteur 1

pastouche.lock_shared();
            Lecture
pastouche.unlock_shared();
```

```
//Lecteur 2

pastouche.lock_shared();
            Lecture
pastouche.unlock_shared();
```

```
//Lecteur 3

pastouche.lock_shared();
            Lecture
pastouche.unlock_shared();
```

Spinlock

Même principe qu'un mutex à l'exception que :
les threads qui essayent d'acquérir le spinlock ne sont pas préemptés
ils attendent la libération du verrou.

Pas officiellement implémenté en C++

Existe en POSIX

Sortie du spinlock immédiate quand le verrou est libéré

Gains/Chutes de performances élevé(e)s (+/- 2x)

Conserver des threads dormants sur le CPU
= ressource inaccessible aux threads actifs

Mécanisme peut être risqué en fonction de son implémentation

Les opérations atomiques (1)

#include <atomic>

Ensemble de primitives permettant de garantir des accès atomiques **à une ressource unique** (une variable basique)

Pas besoin de primitive de synchronisation

Plus rapide qu'un mutex

Cependant… il peut être nécessaire de définir l'ordonnancement des accès mémoires pour utiliser les opérations atomiques correctement

Les opérations atomiques (2)

```
17  #include <atomic>
18
19  std::atomic<int> a = 0;
20
21  void worker(int n)
22  {
23          a.fetch_add(400);
24          a.fetch_sub(200);
25          a++;
26          a--;
27  }
28
29
30  int main()
31  {
32      a.store(0);
33
34      std::threads t1(worker);
35      std::threads t2(worker);
36      ...
37      std::cout << a.load() << std::endl;
```

Lecture / écriture : `load` / `store`

Arithmétique : `fetch_add` / `fetch_sub` / `++` / `--`

Bit à bit : `fetch_and` / `fetch_or` / `fetch_xor`

Compare-and-swap : `compare_exchange_strong/weak`

Booléen simple : `atomic_flag::test_and_set` / `clear`

Memory_order (1)

CPU et compilateur peuvent **réordonner les instructions pour optimiser la performance.**

= l'ordre de ces instructions ne sera pas forcément respecté - >

```
x++;
b++;
c--;
```

Avec std::atomic l'ordre sera préservé sauf si, par souci de performance on spécifie un autre **memory_order** que celui par défaut

memory_order = *"comment les opérations avant/après doivent être visibles pour les autres threads"*

Permet éventuellement une légère amélioration des performances

Memory_order (2)

memory_order_relaxed	Atomicité seulement. Pas de synchronisation mémoire. Performances maximales
memory_order_seq_cst (par défaut)	Séquentielle strictement cohérente. Toutes les opérations apparaissent **dans le même ordre pour tous les threads**.
memory_order_acquire	Pour une lecture : aucune opération suivante ne peut être déplacée **avant**. Utile pour consommer des données produites par un autre thread.
memory_order_release	Pour une écriture : aucune opération précédente ne peut être déplacée **après**. Utile pour publier des données.
memory_order_acq_rel	Pour compare_exchange_strong() & compare_exchange_weak()
memory_order_consume	*Rarement utilisé, support faible. OSEF*

Memory_order (3)

```cpp
struct Data
{
    int a;
    int b;
    int c;
};

Data data;
std::atomic<bool> ready = false;

void prod()
{
    data.a = 42;
    data.b = 99;
    data.c = 147;

    ready.store(true, std::memory_order_relaxed);
}

void cons() {
    while (!ready.load(std::memory_order_relaxed)) {
        // spin
    }
    // Peut lire des valeurs incohérentes !
    std::cout << data.a << " " << data.b << " " << data.c << std::endl;
}
```

release

acquire

```cpp
int main() {
    std::thread t1(prod);
    std::thread t2(cons);

    t1.join();
    t2.join();
}
```

Console de débogage Microso

```
42 99 0
```

CAS : Compare And Swap

```
std::atomic<int> counter{5};
int expected = 5;
bool success = counter.compare_exchange_strong(expected, 10);

//success = true    counter = 10    expected = 5
```

```
std::atomic<int> counter{8};
int expected = 5;
bool success = counter.compare_exchange_strong(expected, 10);

//success = false   counter = 8     expected = 8
```

Pour améliorer les perfs -> un 3e paramètre -> **memory_order_acq_rel**

compare_exchange_weak() est plus rapide mais :

- peut échouer et renvoyer *false* alors que cela devrait être *true*
- à utiliser en boucles

TP - Data race

Veuillez finaliser le code disponible sur eCampus

```
--- N = 10000000 and Threads = 4
40000000 (       us) - sequential (Thread)
30000000 (       us) - datarace (no bug if no luck)
39978011 (       us) - forced datarace (no bug if no luck)
40000000 (       us) - atomic (works only with a very simple case)
40000000 (       us) - mutex
40000000 (       us) - mutex lock guard
40000000 (       us) - spinlock
40000000 (       us) - mutex all (nearly sequential execution path)
40000000 (       us) - mutex lock guard all (nearly sequential execution path)
40000000 (       us) - spinlock all (nearly sequential execution path)

--- N = 1000000 and Threads = 40
40000000 (       us) - sequential (Thread)
40000000 (       us) - datarace (no bug if no luck)
37834909 (       us) - forced datarace (no bug if no luck)
40000000 (       us) - atomic (works only with a very simple case)
40000000 (       us) - mutex
40000000 (       us) - mutex lock guard
40000000 (       us) - spinlock
40000000 (       us) - mutex all (nearly sequential execution path)
40000000 (       us) - mutex lock guard all (nearly sequential execution path)
40000000 (       us) - spinlock all (nearly sequential execution path)

40000000 (       us) - sequential (MAIN)
```

TP Matrix 2

Veuillez réaliser un programme qui :

1. Fait la même chose que l'exercice 1
2. Le calcul est cette fois réalisé par des threads
3. **Instanciation d'un thread par case de la matrice C**

Conservez bien votre programme, car vous allez en avoir besoin plus tard (dossier à réaliser)

TP Matrix 3

Veuillez réaliser un programme qui :

1. Fait la même chose que l'exercice 1
2. Le calcul est cette fois réalisé par des threads
3. Instanciation d'un thread par ligne de la matrice C

Conservez bien votre programme, car vous allez en avoir besoin plus tard (dossier à réaliser)

TP Matrix 4 (A et B)

Veuillez réaliser un programme qui :

1. Fait la même chose que l'exercice 1
2. Le calcul est cette fois réalisé par des threads
3. Le **nombre de threads** à instancier est demandé à l'utilisateur

[0.25T, 0.5T, 0.75T, T, 1.5T, 2T] *où T= Threads de votre CPU*

Version A du TP : **distribution alternée des threads :**

T0	T1	T2	T3	T0
T1	T2	T3	T0	T1
T2	T3	T0	T1	T2
T3	T0	T1	T2	T3
T0	T1	T2	T3	T0

Version A du TP : **distribution contiguë des threads :**

T0	T0	T0	T0	T0
T0	T1	T1	T1	T1
T1	T1	T2	T2	T2
T2	T2	T2	T3	T3
T3	T3	T3	T3	T3

Conservez bien votre programme, car vous allez en avoir besoin plus tard (dossier à réaliser)

Multithreading en C++

SYNCHRONISATIONS

Sémaphore à compte (1)

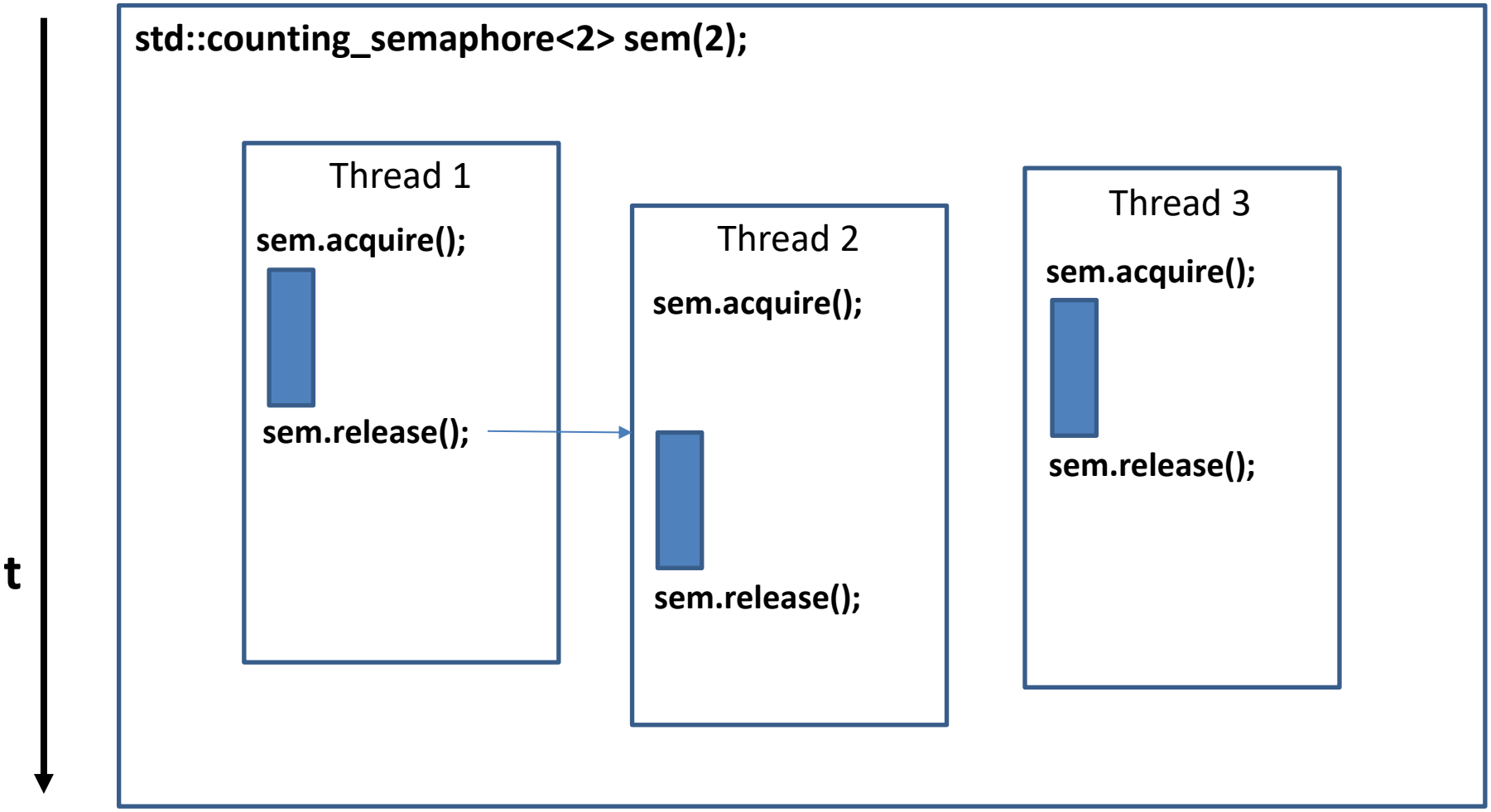
Le mutex partagé ne permet pas de limiter le nombre d'acquisitions

Le sémaphore oui car il est composé d'un compteur et d'une file de tâches en attente.

le compteur est initialisé avec le nombre d'éléments de ressources initialement disponible.

La prise d'un sémaphore dont le compteur est négatif ou nul bloque l'appelant

Sémaphore à compte (2)



Sémaphore à compte (3)

```cpp
#include <iostream>
#include <thread>
#include <semaphore>

#define NOMBRE 10

int tableau[NOMBRE];
std::counting_semaphore<NOMBRE> semEcrire(NOMBRE);
std::counting_semaphore<NOMBRE> semLire(0);

void fprod()
{
    int cpt = 0, donnee=0;
    for (int i = 0; i < NOMBRE * 2; i++)
    {
        donnee = rand()%1024;
        printf("Ecriture[%d]=%d\n", i, donnee);

        semEcrire.acquire();
        tableau[cpt] = donnee;
        semLire.release();

        cpt++;
        if (cpt >= NOMBRE) cpt = 0;
    }
}
```

```cpp
void fcons()
{
    int cpt = 0, donnee = 0;
    for (int i = 0; i < NOMBRE * 2; i++)
    {
        semLire.acquire();
        donnee= tableau[cpt];
        semEcrire.release();

        printf("\tLecture[%d]=%d\n", i, donnee);
        cpt++;
        if (cpt >= NOMBRE) cpt = 0;
    }
}
```

```cpp
int main()
{
    std::thread tprod(fprod);
    std::thread tcons(fcons);

    tprod.join();
    tcons.join();

    return 0;
}
```

```
Ecriture[0]=3736
Ecriture[1]=1020
        Lecture[0]=3736
        Lecture[1]=1020
Ecriture[2]=9713
        Lecture[2]=9713
Ecriture[3]=8494
        Lecture[3]=8494
Ecriture[4]=9066
        Lecture[4]=9066
Ecriture[5]=8903
        Lecture[5]=8903
Ecriture[6]=2985
        Lecture[6]=2985
Ecriture[7]=7693
        Lecture[7]=7693
Ecriture[8]=8650
Ecriture[9]=8927
Ecriture[10]=9811
        Lecture[8]=8650
        Lecture[9]=8927
Ecriture[11]=2889
Ecriture[12]=5114
        Lecture[10]=9811
Ecriture[13]=8038
        Lecture[11]=2889
        Lecture[12]=5114
```

Sémaphore binaire

Un sémaphore binaire peut être utilisé pour protéger une section critique comme avec un mutex

Mais … c**e n'est pas son utilité !**

	Sémaphore binaire	Mutex
Utilité	Mécanisme de synchronisation	Protéger une section critique
Qui peut release/unlock ?	N'importe quel thread	Uniquement le thread qui a acquis le mutex

TP - semaphore

- **3 threads "writers"** qui écrivent en boucle minimum 20 valeurs dans le buffer
- **2 threads "readers"** qui lisent en permanence les nouvelles valeurs écrites dans le buffer
- Quand les "writers" ont terminé, le "main" informe les "readers" qu'ils doivent se couper une fois toutes les valeurs lues.
- Ressources partagées :
  - un **buffer circulaire** implémenté avec un tableau statique de 10 int
  - la position courante d'écriture dans le buffer
  - la position courante de lecture dans le buffer
  - une **variable atomique** qui est le nombre de cases non encore lues

```
Reader 11 : Buffer[3]=59
Reader 11 : Buffer[4]=41
Reader 11 : Buffer[5]=43
Writer 2 : Buffer[6]=51
Writer 2 : Buffer[7]=38
Writer 2 : Buffer[8]=27
Writer 2 : Buffer[9]=60
Reader 11 : Buffer[6]=51
Reader 11 : Buffer[7]=38
Reader 11 : Buffer[8]=27
Reader 11 : Buffer[9]=60
Reader 11 job done !
Reader 12 : Buffer[7]=60
Reader 12 job done !
```

Attente d'un signal

Quand un thread doit attendre un signal avant de poursuivre :

```
3   bool go=false;
4
5   void t1()
6   {
7       //job job
8       while(!go) // wait t2
9       {
10          this_thread::sleep_for(chrono::miliseconds(1));
11      }
12      //job job
13  }
14
15  void t2()
16  {
17      //job job
18      go = true;
19  }
```

Pas top : gaspillage CPU

Idée: mettre le thread au repos en attente du signal -> **variable conditionnelle**

Variable conditionnelle

```
3   std::mutex m;
4   std::condition_variable cv;
5   bool go=false;
6
7   void t1()
8   {
9       //job job
10      std::unique_lock<std::mutex> lock(m);
11      while(!go)
12      {
13          cv.wait(lock);
14      }
15      //job job
16  }
17
18  void t2() {
19
20      m.lock();
21      go = true;
22      m.unlock();
23      cv.notify_one();
24  }
```

Mutex (obligatoire)

Variable conditionnelle

Flag (~obligatoire)

Verrouille le mutex

Libère temporairement le mutex le temps du dodo

Libération automatique de *m*

```
while(!go)
{
    cv.wait(lock);
}
```

Les "pros" utilisent plutôt cette syntaxe :

```
cv.wait(lock, [] { return go; });
```

Lambda

Variable conditionnelle : pièges

Techniquement, on peut se passer du flag… mais :

```
void t1()
{
    //job job
    std::unique_lock<std::mutex> lock(m);
    cv.wait(lock);
    //job job
}
```

Spurious wakeup :
Il arrive que cv.*wait* soit déclenché alors qu'aucune notification n'a été émise

Notification manquée :
En l'absence de flag, si la notification de T2 est envoyée avant que T1 soit en cv.wait
T1 ne recevra jamais la notification et restera en attente

```
std::unique_lock<std::mutex> lock(m);
while(!go)
{
    cv.wait(lock);
}
```

L'ajout du test sur le flag permet d'éviter ces 2 pièges

Variable conditionnelle : bonus

```
bool ok = cv.wait_for(lock, chrono::milliseconds(500), [] { return go; });
if(ok)
    // job job
else
    // timeout reached
```

```
bool ok = cv.wait_until(lock, chrono::steady_clock::now() + 1s, [] { return go; });
if(ok)
    // job job
else
    // timeout reached
```

```
cv.notify_one();     //réveille un thread en attente

cv.notify_all();     //réveille tous les threads en attente
```

Attention au mutex (voir prochain slide)

notify_all et gestion du mutex (1/2)

```
3   std::mutex m;
4   std::condition_variable cv;
5   bool go=false;
6
7   void tworkers()
8   {
9       //job job 1
10      std::unique_lock<std::mutex> lock(m);
11      while(!go)
12      {
13          cv.wait(lock);
14      }
15      //job job 2
16  }
17
18  void tboss() {
19
20      m.lock();
21      go = true;
22      m.unlock();
23      cv.notify_all();
24  }
```

Les "tworkers" ne pourront pas travailler simultanément !

le mutex sera libéré en fin de *scope*

notify_all et gestion du mutex (2/2)

```
std::mutex m;
std::condition_variable cv;
bool go=false;

void tworkers()
{
    //job job 1
    std::unique_lock<std::mutex> lock(m);
    while(!go)
    {
        cv.wait(lock);
    }
    lock.unlock();
    //job job 2
}

void tboss() {

    m.lock();
    go = true;
    m.unlock();
    cv.notify_all();
}
```

Libération manuelle du mutex

***"job job 2"* peut être traiter en parallèle**

TP – variable conditionnelle

- **3 threads** qui élèvent x à la puissance y et affichent le résultat

- **le main** demande en boucle :
  - d'encoder une valeur pour x et y
  - si l'utilisateur veut effectuer un autre calcul ou non

- Un calcul sera systématiquement effectué par un thread réveillé avec notify_one()

- Quand l'utilisateur voudra quitter le programme, les threads se couperont après s'être réveillés par notify_all();

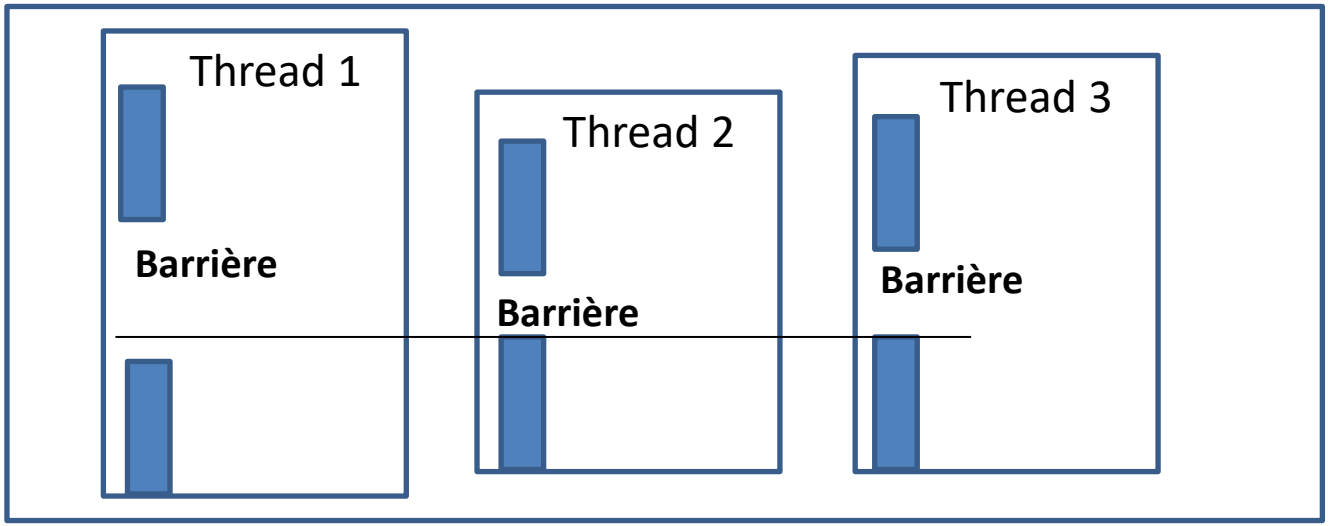
```
x = 4
y = 2
Thread[1] : 4^2 = 16
Autre calcul ? (0=non / 1= oui) 1
x = 8
y = 3
Thread[2] : 8^3 = 512
Autre calcul ? (0=non / 1= oui) 1
x = 7
y = 2
Thread[3] : 7^2 = 49
Autre calcul ? (0=non / 1= oui) 1
x = 5
y = 2
Thread[1] : 5^2 = 25
Autre calcul ? (0=non / 1= oui) 0
Thread[2] : fin du thread
Thread[1] : fin du thread
Thread[3] : fin du thread
```

Barrière de synchronisation

permet de bloquer jusqu'à ce qu'**un nombre fixé d'événements** soit atteint

En C++20 il y a les barrières :

- *std::latch* = à usage unique
- *std::barrier* = réutilisable



std::latch (1)

```cpp
#include <cstdio>
#include <thread>
#include <latch>
#include <vector>

#define N 5

std::latch latch(N);

void fct(int id)
{
    printf("Thread[%d] : Attente des autres\n", id);
    latch.count_down();
    latch.wait();
    printf("Thread[%d] : ok, go dodo\n", id);
}

int main()
{
    std::vector<std::thread> threads;

    for (int i = 0; i < N; ++i) threads.emplace_back(fct, i);

    for (int i = 0; i < N; ++i) threads[i].join();
    return 0;
}
```

Initialisation à N

***latch* -1**

Attente tant que *latch* >0

```
Thread[0] : Attente des autres
Thread[1] : Attente des autres
Thread[2] : Attente des autres
Thread[3] : Attente des autres
Thread[4] : Attente des autres
Thread[3] : ok, go dodo
Thread[1] : ok, go dodo
Thread[4] : ok, go dodo
Thread[0] : ok, go dodo
Thread[2] : ok, go dodo
```

std::latch (2)

```cpp
#include <cstdio>
#include <thread>
#include <latch>
#include <vector>

#define N 5

std::latch latch(N);

void fct(int id)
{
    printf("Thread[%d] : Attente des autres\n", id);
    latch.arrive_and_wait();
    printf("Thread[%d] : ok, go dodo\n", id);
}

int main()
{
    std::vector<std::thread> threads;

    for (int i = 0; i < N; ++i) threads.emplace_back(fct, i);

    for (int i = 0; i < N; ++i) threads[i].join();
    return 0;
}
```

***latch* -1 et attente tant que *latch* >0**

std::latch (3)

```
8   std::latch latch(N);
9   int tab1[N];
10  int tab2[N];
11
12  void fct(int id)
13  {
14      tab1[id] = 42; //fait des super calculs
15      latch.count_down();
16      tab2[id] = 46; //fait d'autres super calculs
17  }
18
19  void show(int* tab)
20  {
21      printf("Tab[ ");
22      for (int i = 0; i < N; ++i) printf("%d ", tab[i]);
23      printf("]\n");
24  }
25
26  int main()
27  {
28      std::vector<std::thread> threads;
29      for (int i = 0; i < N; ++i) threads.emplace_back(fct, i);
30
31      latch.wait();
32      show(tab1);
33
34      for (int i = 0; i < N; ++i) threads[i].join();
35      show(tab2);
36      return 0;
37  }
```

Le *wait* ne doit pas forcément se faire là où il y a le *count_down*

latch -1 et n'attend pas

Attend que tous les threads aient passé le *latch*

```
Tab[ 42 42 42 42 42 ]
Tab[ 46 46 46 46 46 ]
```

std::barrier (1)

Contrairement à *latch* qui est à usage unique, *barrier* est réutilisable

```cpp
#define N 4

std::barrier barrier(N);

void fct(int id)
{
    for (int i = 0; i < 2; ++i)
    {
        printf("Thread[%d] / boucle %d / partie 1 \n", id, i);
        barrier.arrive_and_wait();
        printf("Thread[%d] / boucle %d / partie 2 \n", id, i);
        barrier.arrive_and_wait();
    }
}

int main()
{
    std::vector<std::thread> threads;
    for (int i = 0; i < N; ++i) threads.emplace_back(fct, i);
    for (int i = 0; i < N; ++i) threads[i].join();

    return 0;
}
```

```
Thread[0] / boucle 0 / partie 1
Thread[1] / boucle 0 / partie 1
Thread[2] / boucle 0 / partie 1
Thread[3] / boucle 0 / partie 1
Thread[2] / boucle 0 / partie 2
Thread[1] / boucle 0 / partie 2
Thread[3] / boucle 0 / partie 2
Thread[0] / boucle 0 / partie 2
Thread[3] / boucle 1 / partie 1
Thread[1] / boucle 1 / partie 1
Thread[0] / boucle 1 / partie 1
Thread[2] / boucle 1 / partie 1
Thread[2] / boucle 1 / partie 2
Thread[0] / boucle 1 / partie 2
Thread[1] / boucle 1 / partie 2
Thread[3] / boucle 1 / partie 2
```

std::barrier (2)

```
7   #define N 4
8
9   std::barrier barrier(N);
10
11  void fct(int id)
12  {
13      for (int i = 0; i < 2; ++i)
14      {
15          printf("Thread[%d] / boucle %d / partie 1 \n", id, i);
16          barrier.arrive_and_wait();
17          printf("Thread[%d] / boucle %d / partie 2 \n", id, i);
18          if (id == 0)
19          {
20              barrier.arrive_and_drop();
21              return;
22          }
23          barrier.arrive_and_wait();
24      }
25  }
26
27  int main()
28  {
29      std::vector<std::thread> threads;
30      for (int i = 0; i < N; ++i) threads.emplace_back(fct, i);
31      for (int i = 0; i < N; ++i) threads[i].join();
32
33      return 0;
34  }
```

un thread participant
à la barrière
veut quitter le flux.

C'est possible avec
arrive_and_drop

```
Thread[0] / boucle 0 / partie 1
Thread[1] / boucle 0 / partie 1
Thread[2] / boucle 0 / partie 1
Thread[3] / boucle 0 / partie 1
Thread[2] / boucle 0 / partie 2
Thread[0] / boucle 0 / partie 2
Thread[3] / boucle 0 / partie 2
Thread[1] / boucle 0 / partie 2
Thread[1] / boucle 1 / partie 1
Thread[2] / boucle 1 / partie 1
Thread[3] / boucle 1 / partie 1
Thread[3] / boucle 1 / partie 2
Thread[1] / boucle 1 / partie 2
Thread[2] / boucle 1 / partie 2
```

TP – latch/barrier (1/2)

- 5 threads au code identique qui :
  - génèrent une valeur aléatoire dans un tableau
  - le thread 0 affiche la moyenne des 5 valeurs
  - on recommence en boucle
  - a chaque itération (sauf la première)

    le thread à l'id le plus élevé se coupe

- le programme s'arrête quand il n'y a plus qu'un seul thread

```
tab[1]=1
tab[0]=20
tab[2]=8
tab[3]=13
tab[4]=7
AVG = 9.800000

thread[4] je me casse
tab[1]=11
tab[2]=5
tab[0]=6
tab[3]=13
AVG = 8.750000

thread[3] je me casse
tab[0]=18
tab[2]=2
tab[1]=0
AVG = 6.666667

tab[0]=11
tab[1]=7
thread[2] je me casse
AVG = 9.000000

tab[0]=1
thread[1] je me casse
AVG = 1.000000
```

TP – latch/barrier (2/2)

La fonction rand() n'est pas "**thread-safe**"

code thread-safe *= code utilisable par plusieurs threads simultanément sans provoquer d'erreurs, de corruption de données ou de comportements imprévisibles.*

Solution :

```
void thread_fct(int id)
{
    thread_local std::mt19937 random{ std::random_device{}() };
    std::uniform_int_distribution<int> dist(0, 20);

    int x = dist(random);
```

Variante déterministe :

```
void thread_fct(int id)
{
    thread_local std::mt19937 random(id);
```

Astuces

Les divers mécanismes de synchronisations proposent souvent **des méthodes non-bloquantes** : *try_lock, wait_until, etc.*

Toujours prévoir le nécessaire pour **éviter les *deadlocks*** *: watchdogs, ordrer de verrouillage des mutex, prévoir les départs anticipés des threads, etc.*

Quand vous déboguer, certains bugs ne sont plus visibles car :

- votre code peut être séquentialité par le débogueur
- certaines optimisations compilateur/CPU ne se font pas en mode débogage (par exemple : réagencement des instructions CPU)